2026/2027	D3-2 – TP 4
BTS SIO 2SLAM	Auteur : Elven LECORRE-PENEAU
	Date de rédaction : 06/11/2026

Compte-rendu de TP

TP4 – Votre application android

Choix du thème :

Pour mon application, j'ai décidé de choisir le thème d'une académie de tennis (club de tennis). Je devrais alors gérer des joueurs et des entrainements.

Les critères communs proposés :

C1 – Trois écrans et navigation :

J'ai créé quatre activités, une qui sert d'accueil, une qui sert à remplir un formulaire, l'autre à consulter ce formulaire et enfin une page qui permettra de consulter une liste d'entrainement.
Pour basculer entre les différentes pages je me suis servis de bouton avec des intent, les intents serviront également par la suite pour stocker les informations et les transférer de page en page.

```
buttonConsulterEntrainement = findViewById(R.id.buttonConsulterEntrainement);
buttonFormulaireEntrainement = findViewById(R.id.buttonFormulaireEntrainement);
buttonFormulaireEntrainement.setOnClickListener( View view -> {
    Intent intent = new Intent( packageContext: MainActivity.this, FormulaireEntrainement.class);
    startActivity(intent);
});
```

Voici ci-dessus un exemple pour le bouton d'accès au formulaire d'entrainement.

C2 – Formulaire et validation

J'ai créé un formulaire avec 5 données avec des chaines de texte, des entier et des boutons.

Jeu d'essai :
Coach : Bernard, Balle : Verte, NbParticipants : , Duree : 80, Cours : Cours A.
Coach : Bernard, Balle : Verte, NbParticipants : 8, Duree : 80, Cours : Cours B.
Coach : Bernard, Balle : Verte, NbParticipants : 3, Duree : 180, Cours : Cours C.
Coach : Bernard, Balle : Verte, NbParticipants :4 , Duree : 80, Cours : Cours A.

C3 – Transmission entre Activity :

Je transmets les informations entre le formulaire et sa consultation dans un onglet suivant de la page notamment à l'aide d'extra et de l'intent précédemment créer pour changer de page et avoir une navigation fluide.

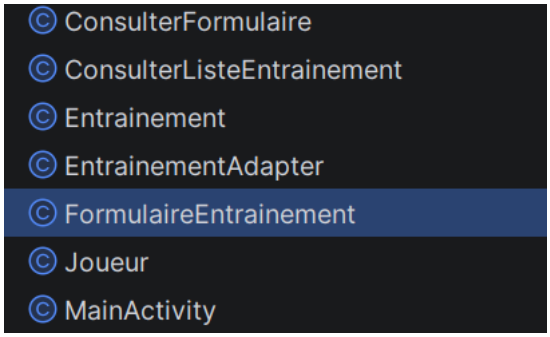
```
intentFormulaire.putExtra(EXTRA_Coach, nomCoach);
intentFormulaire.putExtra(EXTRA_BALLE, nomBalle);
```

```
intent.getStringExtra(FormulaireEntrainement.EXTRA_Coach);
intent.getStringExtra(FormulaireEntrainement.EXTRA_BALLE);
```

C4 – Classe Métier :

J'ai créé deux classes métiers qui ne sont pas toutes utilisé dans l'application pour cause de manque de temps mais les

deux respectent le thème et pourrait être utiliser. Il y a la classe entrainement qui est beaucoup utilisé puis la classe Joueur qui possède également une classe de test dans un package différent. Les deux classes possèdent des attributs, des constructeurs ainsi que les getters/setters.



C5 – Validation dans le modèle

Dans la classe modèle Entrainement j'ai créé des contrôles pour toutes les valeurs avec les différents cas comme les chaines de textes, les entiers et les boutons.

```
if (coach == null || coach.trim().isEmpty())
    throw new IllegalArgumentException("Coach invalide");

if (balle == null || balle.trim().isEmpty())
    throw new IllegalArgumentException("Balle invalide");

if (duree < 30 || duree > 120)
    throw new IllegalArgumentException("Durée invalide");

if (nbParticipants < 1 || nbParticipants > 7)
    throw new IllegalArgumentException("Nombre participants invalide");
```

C6 – Collection et affichage :

J'ai créé une collection entrainements avec des objets de type Entrainement afin de les lires dans la RecyclerView prévu à cet effet. Voici ci-dessous la création de la collection d'objet Entrainement.

```
Entrainement entrainement1 = new Entrainement( coach: "Bernard", balle: "Rouge", nbParticipants: 5, duree: 60, cours: "Cours A");
Entrainement entrainement2 = new Entrainement( coach: "Tristan", balle: "Jaune", nbParticipants: 6, duree: 70, cours: "Cours B");
Entrainement entrainement3 = new Entrainement( coach: "Tristan",  balle: "Mousse", nbParticipants: 5, duree: 40, cours: "Cours C")
ArrayList<Entrainement> entrainements = new ArrayList<>();
entrainements.add(entrainement1);
entrainements.add(entrainement2);
entrainements.add(entrainement3);
```

Les défis aux choix :

D1 – RecyclerView :

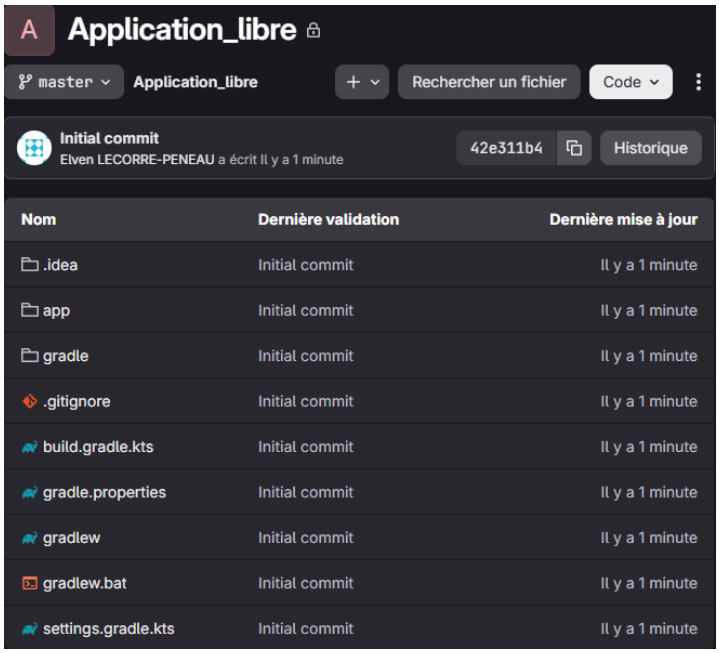
Pour que ça soit plus pratique et qu'on puisse lire des objets de type Entrainement j'ai créé une RecyclerView avec tous les fichiers nécessaire à une RecyclerView comme vu en cours. De plus, selon le cours de l'entrainement la couleur est différente pour différencier les cours. Avec ci-dessous l'adapter de la recyclerView. Puis un exemple de l'affichage.

```
RecyclerView liste = findViewById(R.id.RecyclerViewEntrainement);
liste.setLayoutManager(new LinearLayoutManager( context: this));
EntrainementAdapter adapter = new EntrainementAdapter(entrainements);
liste.setAdapter(adapter);
```



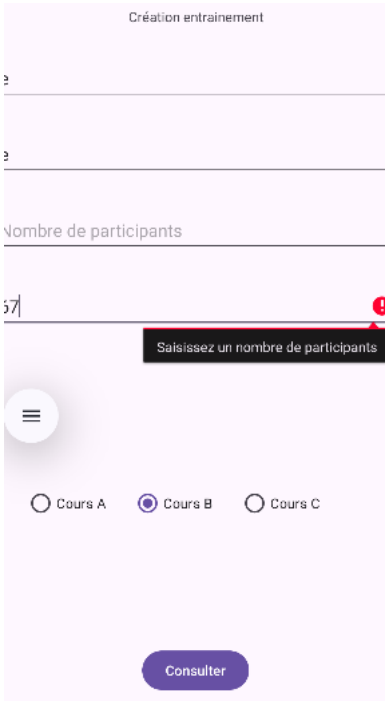
D6 – Historique Git :

Tout d'abord j'ai initialisé un dépôt distant git pour mon projet afin de permettre l'évolution du projet, à chaque feature un commit et un push sera réalisé.



D7 – Vue propre :

Mes différentes vues créer durant l'activité possède plusieurs composants comme la vue du formulaire par exemple qui possède plusieurs EditText, des radioButton ainsi que des TextView.



D8 – Tests :

J'ai créé une classe TestJoueur qui sert à tester si le modèle métier Joueur fonctionne bien et que les limites imposées sont respectées.

```
public class TestJoueur {                                          ⚠ 3  ✓ 55
    public void executerTest() {
        Log.d( tag: "application_ouverte", msg: "test 100 ans " + joueur100ans.getJoueur());
    } catch (IllegalArgumentException e) {
        Log.d( tag: "application_ouverte", msg: "Erreur: Age refusé (100ans)");
    }

    try {
        Joueur joueur101ans = new Joueur( Nom: "Vieux", Prenom: "Harry", Age: 101, NbEntrainement: 2);
        Log.d( tag: "application_ouverte", msg: "test 101 ans " + joueur101ans.getJoueur());
    } catch (IllegalArgumentException e) {
        Log.d( tag: "application_ouverte", msg: "Erreur: Age refusé (101ans)");
    }
```

D9 – Exception :

Enfin, dans les classes métier chaque attribut possède des exceptions en cas de problème avec celui-ci, ou de format non valide. Ici l'exemple de la classe entrainement et de certains de ses attributs.

```
if (coach == null || coach.trim().isEmpty())
    throw new IllegalArgumentException("Coach invalide");

if (balle == null || balle.trim().isEmpty())
    throw new IllegalArgumentException("Balle invalide");

if (duree < 30 || duree > 120)
    throw new IllegalArgumentException("Durée invalide");

if (nbParticipants < 1 || nbParticipants > 6)
    throw new IllegalArgumentException("Nombre participants invalide");
```

Conclusion :

L'application est fonctionnelle avec quelques fonctionnalités. Il pourrait y avoir à l'avenir plusieurs améliorations notamment en utilisant plus la classe Joueur, en attribuant des joueurs à des entrainements, en consultant des entrainements spécifique et bien d'autres encore.